

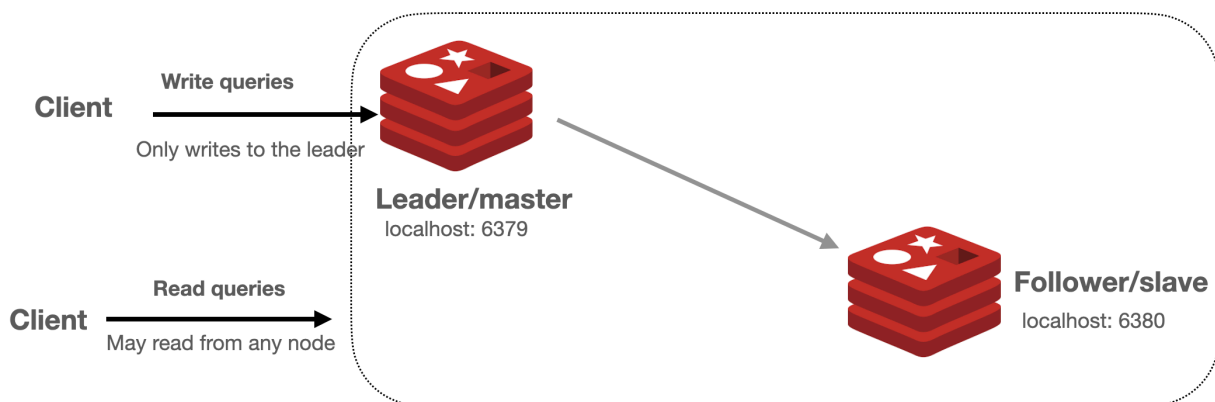
A Hands-On Tutorial for Setting Up a Leader-Follower Cluster

 systemdesignschool.io/fundamentals/database-replication-codelab



Data Replication Tutorial

Let's do a quick demo to see this in action. We will set up a Redis cluster on your local machine with one leader and one follower.



Replication with Redis

Start the primary Redis instance on port 6379 (default port).

And then start the replica Redis instance on port 6380, and specify the primary instance as its leader:

```
redis-server --port 6380 --replicaof 127.0.0.1 6379
```

Replication from Leader to Follower

And run the following Python code to write to the primary and read from both primary and replica.

```
import redis # Connect to the primary and replica nodes
primary = redis.StrictRedis(host='localhost', port=6379, db=0)
replica = redis.StrictRedis(host='localhost', port=6380, db=0)
# Write to the primary node
primary.set('key', 'value')
# Read from the primary node
print("Primary node value:", primary.get('key'))
# Read from the replica node
print("Replica node value:", replica.get('key'))
```

You should see

```
Primary node value:value Replica node value:value
```

Even though we only wrote to the primary node, the replica node still got the changes because it received the data changes from the primary.

We can also see the replica syncing with the primary when it started.

```
:S 26 Apr 2023 17:10:42.924 * PRIMARY <-> REPLICA sync started 11333:S 26 Apr
2023 17:10:42.925 * Non blocking connect for SYNC fired the event. 11333:S 26
Apr 2023 17:10:42.925 * Primary replied to PING, replication can continue...
11333:S 26 Apr 2023 17:10:42.925 * Trying a partial resynchronization (request
28d99e54a99142d4d9de07c45f2d0e82eea125ef:2457). 11333:S 26 Apr 2023 17:10:42.925
* Successful partial resynchronization with primary. 11333:S 26 Apr 2023
17:10:42.925 * PRIMARY <-> REPLICA sync: Primary accepted a Partial
Resynchronization
```

Read From Any Node

Typically in a single-leader setup,

- Read requests can go to any node
- Write requests only go to the primary

In the example above, if you try `replica.set('key', 'value')`, you'll get an error:

```
File "/Library/Frameworks/Python.framework/Versions/3.10/lib/python3.10/site-
packages/redis/connection.py", line 882, in read_response
raise response
redis.exceptions.ReadOnlyError: You can't write against a read only replica.
```

Failover

Let's briefly shut down the leader and turn it back up and see what happens. In the real world, this can be a network partition between the leader and the follower.

```
11550:S 26 Apr 2023 20:22:20.879 # PRIMARY timeout: no data nor PING received...
11550:S 26 Apr 2023 20:22:20.879 # Connection with primary lost. 11550:S 26 Apr
2023 20:22:20.879 * Caching the disconnected primary state. 11550:S 26 Apr 2023
```

```
20:22:20.879 * Reconnecting to PRIMARY 127.0.0.1:6379 11550:S 26 Apr 2023
20:22:20.880 * PRIMARY <-> REPLICATION sync started 11550:S 26 Apr 2023 20:22:20.880
* Non blocking connect for SYNC fired the event. 11550:S 26 Apr 2023
20:22:20.881 * Primary replied to PING, replication can continue... 11550:S 26
Apr 2023 20:22:20.881 * Trying a partial resynchronization (request
28d99e54a99142d4d9de07c45f2d0e82eea125ef:7966). 11550:S 26 Apr 2023 20:22:20.881
* Successful partial resynchronization with primary.
```

We can see the follower attempting to reconnect and trying to synchronize (SYNC event) with the leader as soon as it's able to reach it.